

Query Plans and Index Design

Twenty tickets on the screen.
How much work produced them?

Day 1: follow the rows through a plan

Day 2: test a different access path

The workload has a precise answer

```
SELECT ticket_id, subject, opened_at
FROM performance_demo.tickets
WHERE status = 'in_progress'
ORDER BY opened_at DESC
LIMIT 20;
```

First five IDs: 99906, 99905, 99904, 99903, 99902

One row per ticket. Preserve all twenty ordered IDs in the before/after comparison.

A larger fixture makes search work visible

Status	Tickets	Share
closed	90,000	90%
in_progress	5,000	5%
new	3,000	3%
open	2,000	2%

100,000 synthetic tickets. Demo: performance_demo. Lab: performance_lab.
The setup refreshes statistics before either experiment.

EXPLAIN estimates; ANALYZE executes

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT ticket_id, subject, opened_at
FROM performance_demo.tickets
WHERE status = 'in_progress'
ORDER BY opened_at DESC
LIMIT 20;
```

Plain EXPLAIN: the proposed plan and estimates.

With ANALYZE: observed rows and execution measurements.

These are trusted SELECTs on synthetic data. ANALYZE on a write would perform that write.

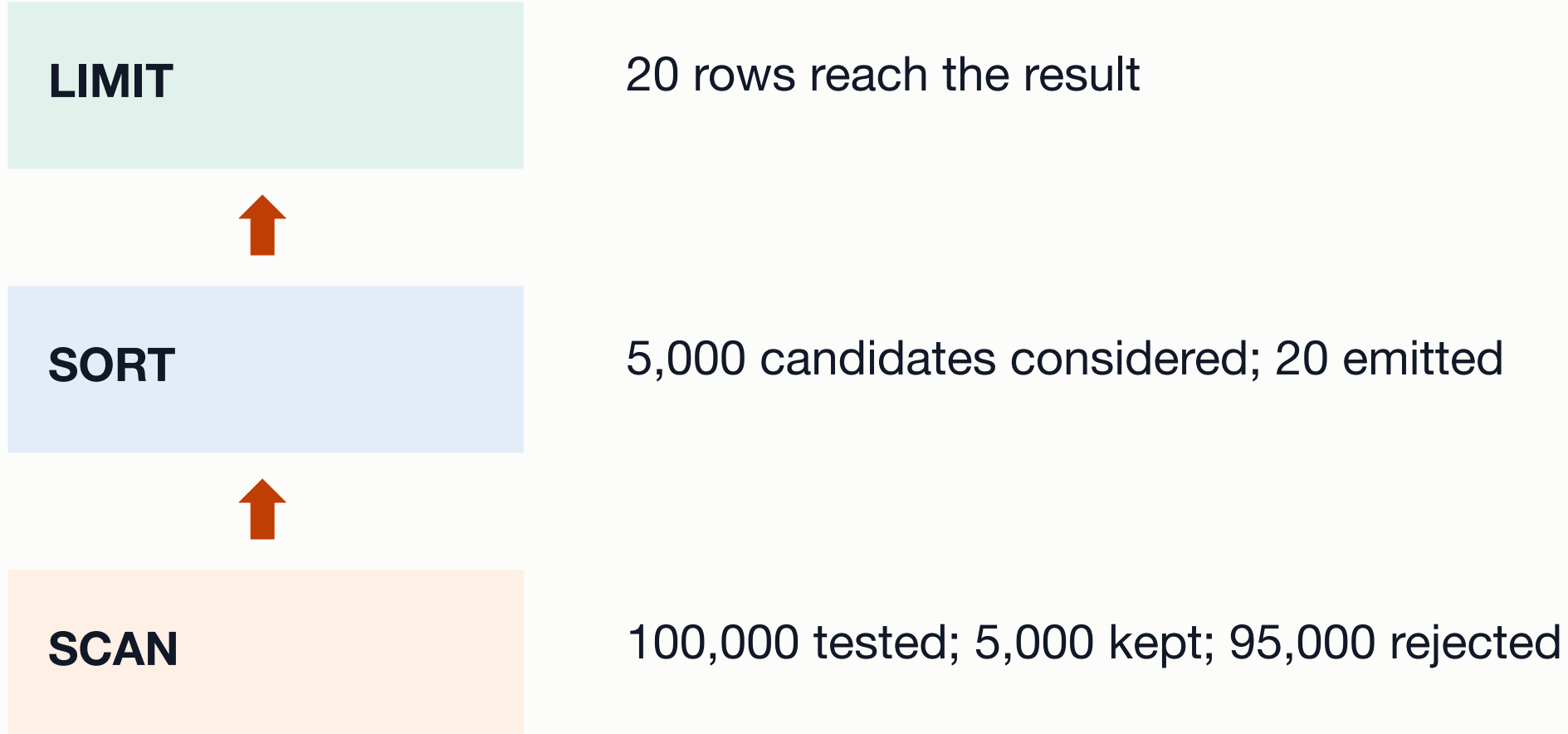
The captured plan returns twenty after a much larger scan

```
Limit (actual rows=20 loops=1)
  -> Sort (actual rows=20 loops=1)
        Sort Key: opened_at DESC
        Sort Method: top-N heapsort
        -> Seq Scan on tickets (actual rows=5000 loops=1)
              Filter: status = 'in_progress'
              Rows Removed by Filter: 95000
```

Sort output: 20 rows. Sort input: 5,000 candidates.

Abbreviated local PostgreSQL 15 capture, September 7, 2026. Your plan may differ.

Rows enter and leave each operator



Predictions and observations use different fields

Plan field	Meaning	Demo scan
rows=...	Estimated output if the node completes	4993
actual rows=...	Observed output per loop	5,000
loops=...	Number of executions	1
cost=startup..total	Planner units, not milliseconds	0 .. 2441

A close row estimate does not create an index. Statistics describe the data; an index provides another path.

Loops and buffers need context

Field	Interpretation
actual rows=3, loops=100	About 300 rows over all loops. Illustrative, not this scan.
shared hit	Requested page found in PostgreSQL's buffer cache
shared read	Page requested below that cache; possibly in the OS cache
Parent counters and timing	Include child work. Do not sum the entire tree.

Rows are not pages. Page-access counters are not unique-row counts.

Selectivity describes how much of the table matches

$$5,000 / 100,000 = 0.05 = 5\%$$

All in-progress tickets / all tickets

Condition	Share of fixture	What it suggests
in_progress	5%	A suitable selective path may avoid much unrelated work
open	2%	Your lab tests another small subset
closed	90%	Returning most rows may favor a broad scan

The existing primary-key index serves a different query

```
SELECT subject FROM performance_demo.tickets  
WHERE ticket_id = 99906;
```

Result: Synthetic ticket 99906

Local plan: Index Scan using the primary-key index

A unique-ID lookup and a newest-by-status queue need different access patterns.

Other operators connect to SQL you already know

Operator	Work performed	Earlier SQL connection
Nested Loop	For each outer row, find matching inner rows	Join tickets to a related row
Hash Join	Build a key lookup, then probe it with another input	Match requester identifiers
Aggregate	Consume rows and produce summaries	COUNT tickets GROUP BY status

Read the actual inputs and loops. Node names alone do not establish whether a plan is efficient.

Lab 1: can five output rows still require a broad scan?

Read the open-queue plan in `performance_lab`.

Change only `LIMIT 20` to `LIMIT 5` and compare scan work.

Restore `LIMIT 20` and propose an ordered index path.

Use qualifying and rejected rows to explain the result. Do not create the index yet.

Brightspace: `week_07_plan_reading.sql`. Keep the relevant plan lines and explanation together.

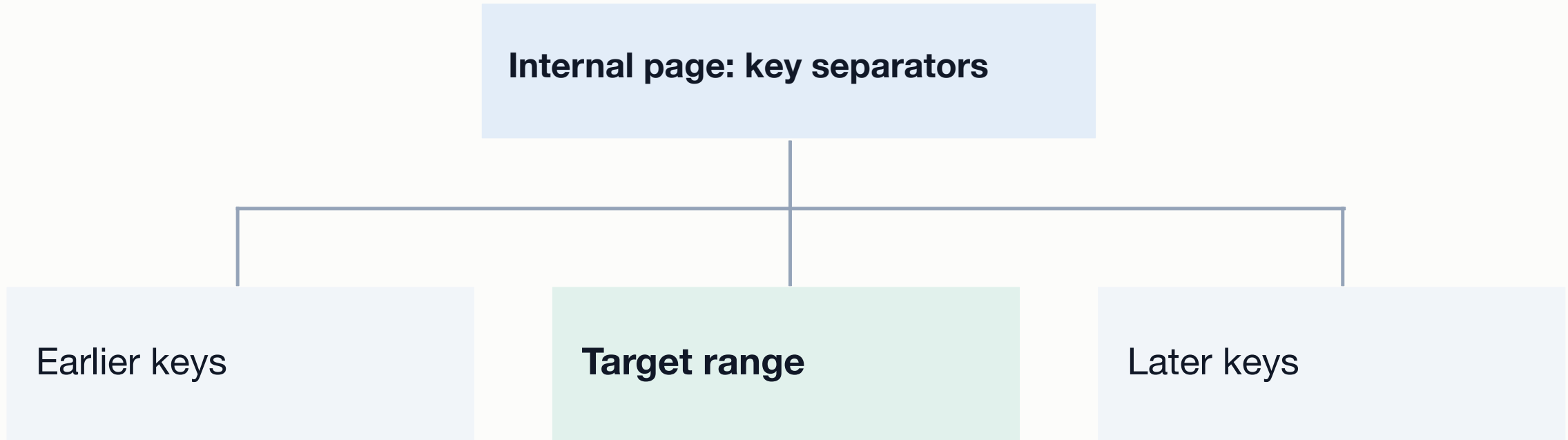
Day 2: another path to the same answer

An ordered structure can change
where the search begins and ends.

Demo: a partial index for in-progress tickets

Lab: a composite index for open tickets

B-tree pages narrow the search range



Simplified model, not physical PostgreSQL pages. Leaf entries lead to table rows and neighboring ordered keys.

A page can point to many child pages, so a large index can remain shallow.

A composite key orders within each leading-key group

(status, opened_at DESC)

status	opened_at (schematic)
closed	September 8, then September 2
in_progress	September 10, then September 1
open	September 9, then September 3

Fix status = 'open': read newest first within that group.
Across groups, the dates are not in one global newest-first order.

The demonstration index contains only in-progress tickets

```
CREATE INDEX inprogress_opened_demo_idx  
ON performance_demo.tickets (opened_at DESC)  
WHERE status = 'in_progress';
```

The index represents 5,000 matching rows. Other statuses remain in the table, outside this index.

Hypothesis: matching entries arrive newest first, without a separate sort. Rerun the identical query.

The measured index scan supplies the requested order

```
Limit (actual rows=20 loops=1)  
  -> Index Scan using inprogress_opened_demo_idx  
      on tickets (actual rows=20 loops=1)
```

Same twenty ordered IDs. No separate Sort.
The parent stops after twenty matching rows.

Abbreviated local PostgreSQL 15 capture, September 7, 2026. Not a forced plan.

A partial index cannot supply rows it excludes

```
SELECT ticket_id, subject, opened_at  
FROM performance_demo.tickets  
WHERE status = 'open'  
ORDER BY opened_at DESC  
LIMIT 20;
```

The index contains in-progress entries, not open entries.
Local plan for this separate query: Seq Scan and Sort.

This tests index coverage. It is not the unchanged query in the before/after comparison.

Storage is measured; write overhead needs its own test

```
SELECT pg_relation_size(  
    'performance_demo.inprogress_opened_demo_idx'  
) AS index_bytes;
```

131,072 bytes in this captured build

Measured here: index space and SELECT behavior.

Not measured here: concurrent write cost or production capacity.

The observations support a mechanism, not a service guarantee

Observation	Before	After
Result IDs	20 recorded IDs	Same ordered IDs
Access / sort	Seq Scan + Sort	Ordered Index Scan
Top-level shared buffers	1191 hits / 0 reads	4 hits / 2 reads
Two execution observations	5.026, 5.063 ms	0.010, 0.005 ms

Captured local synthetic workload, September 7, 2026. Timing, cache state, and sampled estimates can vary.

The comparison keeps the question and data fixed

Same selected columns, predicate, order, and limit.

Same source rows; statistics refreshed before the baseline.

Repeated observations before and after one index addition.

A Limit may stop a child early. Full-range estimates versus twenty requested rows do not, alone, prove bad statistics.

A developer needs a specific recommendation

Keep the partial index for the in-progress queue in this fixture. It preserved all twenty IDs and removed the separate sort. The index occupied 128 KiB in the local build. Before production use, test the actual workload and write costs; this read experiment does not measure them.

A worked example from the demonstration. Your lab recommendation must use your own open-queue results.

Lab 2: test the composite index for the open queue

Reset performance_lab and record the twenty-row baseline.

Create (status, opened_at DESC), then rerun the same query.

Compare IDs, plan work, repeated timings, and index space.

Write a brief developer update: keep, remove, or test further. Distinguish observed benefit from unmeasured costs.

Brightspace: week_07_index_decision.sql. No separate benchmark report.

A useful change preserves the answer and explains the work

Small results can hide large searches.
Index order and coverage must fit the query.
A recommendation includes the cost.

Next week: a backup must restore correctly, just as today's faster query must return the correct tickets.